

1. Visão geral

O ClinSecure utiliza um processo de autenticação em múltiplas etapas, desenvolvido com **Django**, que combina:

- autenticação por usuário e senha;
- autenticação de dois fatores (2FA) utilizando **TOTP**;
- armazenamento de estado temporário durante o processo de autenticação;
- limitação de tentativas;
- atraso progressivo entre tentativas inválidas;
- bloqueio temporário da conta;
- bloqueio independente da etapa de 2FA;
- registro de eventos de autenticação em uma trilha de auditoria;
- encerramento da sessão por logout.

A aplicação utiliza o sistema de autenticação do Django por meio de `AbstractUser`, enquanto a autenticação TOTP é implementada com `django-otp`. O `django-otp` fornece suporte a OTP e aos algoritmos HOTP/TOTP, podendo ser integrado ao mecanismo de autenticação do Django.

2. Fluxo geral de autenticação

O fluxo de autenticação do sistema ClinSecure é realizado em etapas, com o objetivo de garantir que somente usuários devidamente identificados consigam acessar as funcionalidades protegidas da aplicação.

Inicialmente, o usuário acessa a tela de login e informa seu nome de usuário e senha. O sistema verifica se a conta está temporariamente bloqueada. Caso esteja bloqueada, o acesso é negado e o usuário deve aguardar o término do período de bloqueio.

Se a conta não estiver bloqueada, o sistema realiza a validação das credenciais utilizando o mecanismo de autenticação do Django. Caso o usuário ou a senha estejam incorretos, uma nova tentativa é registrada, o sistema aplica um atraso progressivo e, após atingir o limite de tentativas definido, a conta é bloqueada temporariamente.

Quando o usuário e a senha estão corretos, o sistema ainda não considera a autenticação concluída. Nesse momento, o identificador do usuário é armazenado temporariamente na sessão por meio da variável `pre_otp_user_id`. Essa informação é utilizada para identificar o usuário que está aguardando a validação do segundo fator de autenticação.

Em seguida, o sistema verifica se o usuário possui um dispositivo de autenticação TOTP configurado e confirmado. Caso possua, o usuário é direcionado para a tela de validação do código 2FA. O código informado é comparado com o código gerado pelo aplicativo

autenticador. Se o código estiver incorreto, a tentativa é registrada, sendo aplicado atraso progressivo e, após o limite de tentativas, um bloqueio temporário específico para o 2FA.

Caso o código TOTP esteja correto, o sistema conclui o processo de autenticação utilizando o mecanismo de sessão do Django. Somente nesse momento o usuário passa a ser considerado autenticado e pode acessar as áreas protegidas do sistema.

Caso o usuário ainda não possua o segundo fator configurado, o sistema direciona para a etapa de configuração do 2FA. Nessa etapa é gerado um QR Code contendo as informações necessárias para cadastrar o dispositivo em um aplicativo autenticador. Após o usuário realizar a configuração e informar um código válido, o dispositivo é confirmado e o processo de autenticação é concluído.

Dessa forma, o fluxo completo pode ser resumido nas seguintes etapas:

1. Usuário acessa a tela de login.
2. Usuário informa nome de usuário e senha.
3. O sistema verifica se a conta está bloqueada.
4. O sistema valida as credenciais.
5. Se as credenciais estiverem incorretas, a tentativa é registrada e são aplicados mecanismos de proteção contra força bruta.
6. Se as credenciais estiverem corretas, o sistema armazena temporariamente o identificador do usuário.
7. O sistema verifica a configuração do segundo fator de autenticação.
8. Caso o 2FA esteja configurado, o usuário informa o código TOTP.
9. O sistema valida o código informado.
10. Se o código estiver incorreto, a tentativa é registrada e podem ser aplicados atraso e bloqueio temporário.
11. Se o código estiver correto, o sistema cria a sessão autenticada.
12. O usuário passa a ter acesso às funcionalidades protegidas do sistema.

Esse processo garante que a senha, isoladamente, não seja suficiente para concluir a autenticação. Dessa maneira, o sistema adiciona uma camada adicional de segurança por meio do segundo fator de autenticação e dos mecanismos de proteção contra tentativas automatizadas.

3. Primeira etapa — usuário e senha

O processo começa na função `meu_login_view()`.

O sistema recebe:

- nome de usuário;
- senha.

Esses dados são enviados ao Django através de uma requisição POST.

A autenticação propriamente dita é realizada por:

```
authenticate(  
    request,  
    username=user_name,  
    password=senha  
)
```

Quando as credenciais estão corretas, o sistema não direciona imediatamente o usuário para a página principal.

Primeiro ele verifica se existe um dispositivo TOTP confirmado para aquele usuário.

Justificativa técnica:

Essa decisão é importante porque **senha correta não significa autenticação completa**.

O segundo fator cria uma camada adicional de proteção caso a senha seja:

- descoberta;
- reutilizada;
- vazada;
- obtida através de engenharia social;
- comprometida por outro serviço.

O conceito de 2FA consiste justamente em exigir, além da senha, uma segunda forma de autenticação.

4. Não criação imediata da sessão autenticada

Um dos pontos mais importantes da arquitetura é o uso da variável:

```
request.session['pre_otp_user_id'] = usuario.id
```

Nesse momento, o sistema guarda **temporariamente a identificação do usuário que passou pela primeira etapa**, mas ainda não conclui o login.

Isso permite que a aplicação saiba:

"Este usuário já passou pela validação de usuário e senha e agora precisa completar o segundo fator."

O código somente chama:

```
login(request, usuario)
```

depois da validação do TOTP.

Justificativa técnica :

Essa separação é fundamental para evitar que o usuário seja considerado completamente autenticado antes da validação do segundo fator.

A documentação do django-otp descreve justamente a possibilidade de realizar a autenticação em duas etapas, identificando primeiro o usuário e posteriormente verificando o dispositivo OTP.

5. Verificação do dispositivo 2FA

Após a senha correta, o sistema procura um dispositivo TOTP confirmado:

```
TOTPDevice.objects.filter(  
    user=usuario,  
    confirmed=True  
)
```

Existindo um dispositivo confirmado:

Login



Senha correta



Existe TOTP confirmado?



SIM



Tela de verificação 2FA

Caso ainda não exista, o sistema encaminha o usuário para:

setup_2fa

6. Configuração do primeiro 2FA

Na primeira configuração, o sistema cria ou recupera um dispositivo TOTP:

```
device, created = TOTPDevice.objects.get_or_create(
    user=usuario,
    name="Celular Principal",
    defaults={'confirmed': False}
)
```

Depois é obtida a URL de configuração:

```
otp_uri = device.config_url
```

Essa URL é transformada em QR Code.

O usuário escaneia o QR Code utilizando um aplicativo autenticador compatível com TOTP.

Depois disso, o aplicativo passa a gerar códigos temporários.

O sistema verifica o código através de:

```
device.verify_token(token_digitado)
```

Se o código estiver correto:

```
device.confirmed = True
device.save()
```

e então o login é concluído.

Justificativa técnica do TOTP

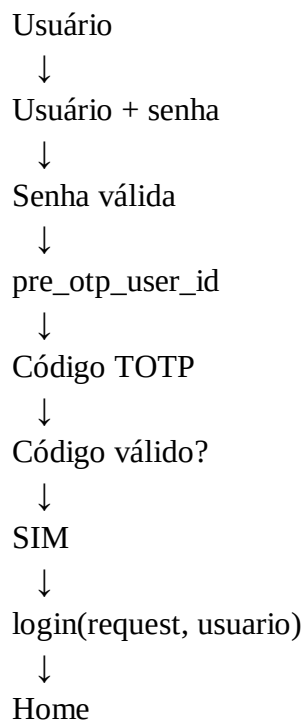
O TOTP é adequado porque o código não é uma senha permanente. Ele é calculado com base em uma chave secreta compartilhada e no tempo atual, fazendo com que o código seja temporário.

Isso reduz o risco associado ao uso de códigos estáticos.

O django-otp possui suporte específico para TOTP e sua integração com Django.

7. Verificação do 2FA em logins posteriores

Depois que o usuário já configurou o 2FA, o fluxo passa a ser:



O código TOTP é validado através de:

`device.verify_token(token_digitado)`

Somente quando essa função retorna sucesso o sistema chama:

`login(request, usuario)`

Depois disso, a variável temporária `pre_otp_user_id` é removida da sessão.

Justificativa

A remoção é importante porque o identificador temporário deixa de ser necessário após a conclusão da autenticação.

Isso evita manter informações temporárias da etapa de autenticação durante a sessão já autenticada.

8. Proteção contra força bruta no login

O sistema também possui uma camada de proteção contra-ataques de força bruta.

Quando usuário ou senha estão incorretos, o contador:

```
usuario_cadastrado.tentativas_login += 1
```

é incrementado.

O sistema também registra a tentativa no AuditLog.

8.1 Atraso progressivo

A aplicação calcula:

```
atraso = usuario_cadastrado.tentativas_login
```

e aplica:

```
time.sleep(atraso)
```

Assim, as tentativas passam a ter atraso progressivo.

Exemplo:

Tentativa	Atraso
1 ^a	1 segundo
2 ^a	2 segundos
3 ^a	3 segundos
4 ^a	4 segundos
5 ^a	bloqueio

Justificativa técnica

O objetivo é reduzir a velocidade com que um atacante consegue testar combinações de credenciais.

Sem esse mecanismo, um atacante poderia realizar uma grande quantidade de tentativas em sequência.

9. Bloqueio após cinco tentativas

Após cinco tentativas incorretas:

```
usuario_cadastrado.bloqueado_ate = (  
    timezone.now() + timedelta(minutes=5)  
)
```

A conta permanece bloqueada durante cinco minutos.

O sistema também registra o bloqueio no AuditLog.

Justificativa técnica

O bloqueio temporário cria uma barreira contra-ataques automatizados.

A escolha de um bloqueio **temporário**, em vez de permanente, também evita que um usuário legítimo fique impedido de acessar a conta indefinidamente.

10. Proteção independente do 2FA

Um ponto muito positivo do projeto é que o 2FA possui seu **próprio contador de tentativas**.

No modelo Usuario existem:

```
tentativas_2fa  
bloqueado_2fa_ate
```

enquanto o login possui:

```
tentativas_login  
bloqueado_ate
```

Isso significa que as duas etapas possuem mecanismos independentes de proteção.

11. Proteção contra tentativa de descobrir o código 2FA

Quando o usuário informa um código 2FA incorreto:

```
usuario.tentativas_2fa += 1
```

O sistema registra a tentativa e aplica atraso progressivo:

```
atraso = usuario.tentativas_2fa  
time.sleep(atraso)
```

Após cinco tentativas:


```
usuario.bloqueado_2fa_ate = (  
    timezone.now() + timedelta(minutes=5)  
)
```

Justificativa técnica

Isso impede que um atacante que tenha descoberto a senha possa testar indefinidamente códigos TOTP.

Essa proteção é especialmente importante porque um código TOTP possui um espaço relativamente pequeno de possibilidades — normalmente seis dígitos.

12. Auditoria

Outro componente importante é o modelo:

```
class AuditLog(models.Model):
```

Ele armazena:

- usuário;
- evento;
- data e hora;
- endereço IP;
- resultado;
- detalhes.

Durante o processo de autenticação são registrados eventos como:

Login bem-sucedido

Login

Bloqueio de conta

Tentativa durante bloqueio

2FA

Bloqueio 2FA

Justificativa técnica

A auditoria permite identificar:

- tentativas de invasão;
- usuários bloqueados;
- tentativas repetidas;
- autenticações bem-sucedidas;
- utilização do 2FA;

- origem das tentativas através do IP.

Isso proporciona **rastreabilidade**, que é particularmente relevante para o ClinSecure, já que o projeto trabalha com informações sensíveis.

O próprio README do projeto estabelece o registro de auditoria como um dos mecanismos de segurança da plataforma.

13. Estrutura do usuário

O projeto utiliza um modelo próprio:

```
class Usuario(AbstractUser):
```

Ou seja, o sistema aproveita a estrutura de autenticação do Django e adiciona informações específicas do projeto.

Entre os campos relacionados à segurança estão:

```
cpf
perfil
dois_fatores_ativado
secret_key
tentativas_login
bloqueado_ate
ultimo_login_falhou
tentativas_2fa
bloqueado_2fa_ate
```

Justificativa técnica

A utilização de AbstractUser evita a necessidade de implementar manualmente mecanismos fundamentais de autenticação, permitindo utilizar recursos consolidados do Django.

Ao mesmo tempo, o modelo é estendido para atender às necessidades específicas do sistema.

14. Controle de acesso às páginas

A página inicial utiliza:

```
@login_required
def home_view(request):
```

Isso significa que somente usuários autenticados podem acessar essa rota.

O logout utiliza:

logout(request)

e posteriormente redireciona o usuário para a tela de login.

Justificativa técnica

A autenticação não deve ser aplicada apenas à tela de login.

É necessário proteger também os recursos que exigem identidade autenticada.

Assim:

Usuário não autenticado



Página protegida



BLOQUEIO

Enquanto:

Usuário autenticado



Página protegida



ACESSO

15. Resumo das camadas de segurança

Mecanismo	Objetivo
Usuário + senha	Primeira etapa de autenticação
Django Authentication	Validação das credenciais
TOTP	Segundo fator de autenticação
pre_otp_user_id	Controlar a etapa intermediária
confirmed=True	Identificar dispositivo 2FA validado
Contador de login	Detectar tentativas repetidas
Delay progressivo	Reduzir velocidade de ataques
Bloqueio de 5 minutos	Impedir tentativas contínuas
Contador independente do 2FA	Proteger a segunda etapa
Bloqueio do 2FA	Evitar tentativa de descoberta do OTP
AuditLog	Rastreabilidade
login_required	Proteger recursos autenticados
logout()	Encerrar a sessão

16. Justificativa Geral da Arquitetura de Autenticação

A arquitetura de autenticação do sistema ClinSecure foi desenvolvida com o objetivo de aumentar a segurança do acesso às informações e funcionalidades da aplicação. Para isso, o sistema utiliza diferentes mecanismos de proteção, formando uma estratégia de defesa em profundidade.

A primeira camada de proteção é composta pela autenticação por usuário e senha. Essa etapa permite verificar a identidade do usuário por meio de suas credenciais. Entretanto, a autenticação não é concluída imediatamente após a validação da senha, pois o sistema exige uma segunda etapa de verificação.

A segunda camada é o uso da autenticação de dois fatores (2FA), implementada por meio do protocolo TOTP. Após informar corretamente usuário e senha, o usuário deve fornecer um código temporário gerado por um aplicativo autenticador. Dessa forma, mesmo que a senha seja descoberta por terceiros, o acesso ao sistema ainda dependerá do segundo fator.

Além da autenticação em duas etapas, o sistema possui mecanismos de proteção contra ataques de força bruta. São utilizadas contagens de tentativas, atrasos progressivos e bloqueios temporários. Esses mecanismos dificultam a realização de diversas tentativas consecutivas de login ou de validação do código 2FA.

O sistema também mantém uma separação entre as tentativas de autenticação por senha e as tentativas de autenticação por 2FA. Essa abordagem permite que cada etapa possua seu próprio controle de tentativas e bloqueio, aumentando a proteção do processo de autenticação.

Outro mecanismo importante é o registro de eventos relacionados à autenticação por meio do sistema de auditoria. Informações como usuário, evento, data, horário, endereço IP e resultado da operação podem ser utilizadas para acompanhar atividades relacionadas ao acesso ao sistema. Esse recurso contribui para a rastreabilidade e facilita a identificação de comportamentos suspeitos.

Por fim, as páginas que exigem autenticação são protegidas utilizando os mecanismos de controle de acesso disponibilizados pelo Django. Dessa maneira, usuários que não possuem uma sessão autenticada não conseguem acessar diretamente determinadas funcionalidades da aplicação.

Portanto, a combinação entre autenticação por senha, autenticação de dois fatores, controle de tentativas, bloqueio temporário, auditoria e proteção das rotas proporciona uma arquitetura de segurança em camadas. Essa abordagem reduz os riscos associados ao comprometimento de credenciais e dificulta tentativas de acesso não autorizado ao sistema.

